

第 1 章作业

1.5 将下列 16 进制数转换为 10 进制数，将 10 进制数转换成 16 进制数

789AH, 0CEFH, 1234D, 7890D

[解析] 789AH=30874D 0CEFH=3311D 1234D=4D2H 7890D=1ED2H

1.6 将下列 10 进制数转换成 8 位二进制补码：

19, 63, -1, -44, 127, -127

[解析] 19=00010011 原=00010011 补

63=00111111 原=00111111 补

-1=10000001 原=11111111 补

-44=10101100 原=11010100 补

127=01111111 原=01111111 补

-127=11111111 原=10000001 补

1.8 已知：

1) $[x_1]_{\text{补}}=0000\ 0001$, $[y_1]_{\text{补}}=1111\ 1111$

2) $[x_2]_{\text{补}}=0101\ 1110$, $[y_2]_{\text{补}}=0011\ 0111$

3) $[x_3]_{\text{补}}=1001\ 1110$, $[y_3]_{\text{补}}=1100\ 0101$

4) $[x_4]_{\text{补}}=0110\ 1110$, $[y_4]_{\text{补}}=1000\ 0100$

请计算 $[x_i]_{\text{补}} + [y_i]_{\text{补}}$, ($i = 1 \sim 4$), 并判断结果是否出现溢出？

[解析] 1) 0000 0000 未溢出

2) 1001 0101 溢出

3) 0110 0011 溢出

4) 1111 0010 未溢出

注：若记符号位向前进位为 CP，次高位向前进位为 CF，当且仅当 $CP \wedge CF=1$ 时，结果发生溢出

1.9 一个无符号的 4 位的 10 进制数，需要使用多少位二进制数才能表示？如果使用非压缩 BCD 码，则需要多少位二进制数才能表示？

[解析]

无符号四位最大十进制数为 9999，介于 8092 - 16184 之间，因此需要 14 位无符号二进制数表示

非压缩 BCD 一个字节仅用来表示一位 BCD 码，其中低 4 位表示 0~9，高 4 位为 0，即需要 8 位表示 4 位，因此需要 $4 \times 8=32$ 位非压缩 BCD 码

[非教材上的习题]

[补充 1.16] 冯·诺依曼结构计算机中数据采用二进制编码表示，其主要原因是 (D)

- I. 二进制的运算规则简单
 - II. 制造两个稳态的物理器件较容易
 - III. 便于用逻辑门电路实现算术运算
- A) 仅 I、II
B) 仅 I、III
C) 仅 II、III
D) I、II、III

[解析] 常识，略。

[补充 1.17] 假定带符号整数采用补码表示，若 int 型变量 x 和 y 的机器数分别是 FFFF FFDFH 和 0000 0041H，则 x 、 y 的值以及 $x-y$ 的机器数分别是 (C)

- A) $x = -65$, $y = 41$, $x-y$ 的机器数溢出
B) $x = -33$, $y = 65$, $x-y$ 的机器数为 FFFF FF9DH
C) $x = -33$, $y = 65$, $x-y$ 的机器数为 FFFF FF9EH
D) $x = -65$, $y = 41$, $x-y$ 的机器数为 FFFF FF96H

[解析]

$$\because 4 \times 16 + 1 = 65, \therefore 0000\ 0041H = 65D$$

可知只有 B 和 C 可能正确

$$x = -33D, y = 65D \Rightarrow x - y = -98D$$

方法 1: 依据补码定义计算，

$$\begin{aligned} & 2^{32} - 98 \\ &= (2^{32} - 1) - 97 \\ &= [(2^{32} - 1) - (2^8 - 1)] + [(2^8 - 1) - 97] \\ &= FFFFFFF00H + 158 \\ &= FFFFFFF00H + (128 + 16 + 8 + 4 + 2) \\ &= FFFFFFF00H + 1001\ 1110B \\ &= FFFFFFF00H + 9EH \\ &= FFFF\ FF9EH \end{aligned}$$

方法 2: 真值取反+1 得到补码

$$98D = 96D + 2D = 6 \times 16 + 2 = 62H = 0110\ 0010B$$

低 7 位取反得，001 1101B

+1 得，001 1110B

故 98D 的补码是，1001 1110B = 9EH

[补充 1.18] 已知带符号整数用补码表示，float 型数据用 IEEE754 标准表示。假定变量 x 的类型只可能是 int 或 float，当 x 的机器数为 C800 0000H 时， x 的值可能是 (A)

- A) -7×2^{27}
B) -2^{16}
C) 2^{17}
D) 25×2^{27}

[解析]

(1) 若 x 是 `int` 类型

$C800\ 0000H = 1100\ 1000\ 0000\dots0000B$

方法 1: 补码定义

因为 x 是补码表示的负数, 故 $1100\ 1000\ 0000\dots0000B = 2^{31} + 2^{30} + 2^{27} = 2^{32} - |x|$

$|x| = 2^{32} - (2^{31} + 2^{30} + 2^{27}) = 2^{27} \times (2^5 - 2^4 - 2^3 - 2^0) = 2^{27} \times 7$

$\Rightarrow x = -7 \times 2^{27}$

方法 2: 真值取反+1 得到补码

除符号位外取反,

$1100\ 1000\ 0000\dots0000B \Rightarrow 1011\ 0111\ 1111\dots1111B$

除符号位外部分-1,

$1011\ 0111\ 1111\dots1B \Rightarrow 1011\ 1000\ 0000\dots0000B$

$x = -(2^{29} + 2^{28} + 2^{27}) = -(2^2 + 2^1 + 2^0) \times 2^{27} = -7 \times 2^{27}$

(2) 若 x 是 `float` 类型

依 FP32 定义, $(-1)^S \times 2^{e-127} \times (1 + M)$

$C800\ 0000H = 1100\ 1000\ 0000\dots0000B \Rightarrow S = 1, e = 2^7 + 2^4 = 128 + 16, M = 0$

故 $x = (-1)^1 \times 2^{64-127} \times (1 + 0) = -2^{17}$

[补充 1.19] 已知带符号整数用补码表示。变量 X, Y, Z 的机器数分别为 FFFDH, FFDFH, 7FFCH, 下列结论中, 正确的是 (D)

- A) 若 X, Y, Z 为无符号整数, 则 $Z < X < Y$
- B) 若 X, Y, Z 为无符号整数, 则 $X < Y < Z$
- C) 若 X, Y, Z 为带符号整数, 则 $X < Y < Z$
- D) 若 X, Y, Z 为带符号整数, 则 $Y < X < Z$

[解析]

若 X, Y, Z 为无符号整数, $FFFDH > FFDFH > 7FFCH \Rightarrow X > Y > Z \Rightarrow$ A 和 B 错误

若 X, Y, Z 为有符号整数, $FFFDH < 0, FFDFH < 0, 7FFCH > 0 \Rightarrow Z$ 最大

$FFFDH = 2^{16} - |X| \Rightarrow |X| = 2^{16} - FFFDH = 2^{16} - (FFFFH - 2H)$

$FFDFH = 2^{16} - |Y| \Rightarrow |Y| = 2^{16} - FFDFH = 2^{16} - (FFFFH - 20H)$

可知 $|X| < |Y| \Rightarrow X > Y$, 故选 D

[补充 1.20] 下列数值中, 不能用 IEEE-754 浮点精确表示的 (A)

- A) 1.2
- B) 1.25
- C) 2.0
- D) 2.5

[解析]

IEEE-754 浮点数, 规格化意味着 23bits 的尾数 (定点小数表示的尾数) 中小数点在最高比特左边, 且隐含了小数点左边的 1。

而 $bit[22] = 2^{-1}, bit[21] = 2^{-2} \dots bit[0] = 2^{-23}$ 。

若一个数可以被表示为类似 $\sum_{i \in \mathbb{Z}} 2^i$ 形式, 就可以被 IEEE-754 精确表示。

用排除法, B、C、D 的小数部分都是 2^{-n} 形式, 一定可以被表示。故选 A。

删除[陈晓辉]: E

进一步解读，令集合 $\Phi = \{-1, -2, \dots, -23\}$ ，令集合 $\Psi = \{-127, -126, \dots, 127, 128\}$
若一个数可以被表示为类似 $\sum_{i=\phi+\psi, \phi \in \Phi, \psi \in \Psi} 2^i$ 形式，就可以被 IEEE-754 精确表示。

[补充 1.21] DeepSeek-V3 使用 FP8（8 位浮点数）来提高计算速度并减少训练期间的显存使用量。FP8 是一种 8 位浮点数表示法，FP8 的详细介绍可以参考 <https://developer.nvidia.com/zh-cn/blog/fp8-precision-performance/>。FP8 采取 E4M3 和 E5M2 两种表示方式，其中 E 代表指数位（Exponent），M 代表尾数位（Mantissa），具体信息如下表所示。

Table 1: Details of FP8 Binary Formats

	E4M3	E5M2
Exponent bias	7	15
Infinities	N/A	$S.11111.00_2$
NaN	$S.1111.111_2$	$S.11111.\{01, 10, 11\}_2$
Zeros	$S.0000.000_2$	$S.00000.00_2$
Max normal	$S.1111.110_2 = 1.75 * 2^8 = 448$	$S.11110.11_2 = 1.75 * 2^{15} = 57,344$
Min normal	$S.0001.000_2 = 2^{-6}$	$S.00001.00_2 = 2^{-14}$
Max subnorm	$S.0000.111_2 = 0.875 * 2^{-6}$	$S.00000.11_2 = 0.75 * 2^{-14}$
Min subnorm	$S.0000.001_2 = 2^{-9}$	$S.00000.01_2 = 2^{-16}$

某 8bits 二进制机器数为 1100 0011B，若其表示的是 E4M3 的 FP8 浮点数，其真值为（
（
（
）。提示：（1）无需计算最后数值，写出表达式即可。（2）按照规格化浮点数格式（上表中 normal 情形）计算。

[解析]

E4M3, bias=7, 1100 0011B, $(-1) * 2^{(8-7)} * (1 + 0.25 + 0.125)$

E5M2, bias=15, 1100 0011B, $(-1) * 2^{(16-15)} * (1 + 0.5 + 0.25)$

作业说明

- ☐ 以下以灰色双删除线标识的不做。
- ☐ *标识的为选作题（可以不做）。

第 1 章

[补充 1.22] *查阅资料，总结第 1 届、第 2 届、第 3 届图灵奖获奖人的获奖原因。

[解析]

- Alan Perlis, ALGOL 语言
- Maurice Wilkes, 受 EDVAC 启发设计了 EDSAC
- Richard Wesley Hamming, 编码、校验码

[补充 1.22] *查阅资料，写出下列缩略语的全称，并总结其主要用途（与 CPU 有何不同）。

缩略语	全称	主要用途
CPU		

GPU		
NPU		
DSP		
MPU		
MCU		

[解析]

缩略语	全称	主要用途
CPU	Central Processing Unit	执行指令，完成指令功能
GPU	Graphic Processing Unit	图形加速计算、通用计算
NPU	Neural Processing Unit	加速神经网络计算
DSP	Digital Signal Processor	数字信号处理算法加速
MPU	Embedded Microprocessor Unit	嵌入式系统的处理器
MCU	MicroController Unit	单片机处理器

第 2 章作业 part1

2.1 计算机体系结构、计算机组成和计算机实现三者之间有什么关系？

[解析]

- 计算机体系结构（computer architecture）关注计算机的基本设计思想和由此产生的逻辑结构。
- 计算机组成（computer organization）关注的是机器中各个功能单元的逻辑设计、物理实现和部件之间的互联组织。
- 计算机实现技术关注更底层的集成电路设计、结构设计、组装工艺、电源技术和冷却措施等。

2.2 影响冯·诺依曼结构计算机性能的主要技术瓶颈是什么？

[解析] 指令和数据共用一条主线，取指和取数据会产生冲突

2.8 寄存器和存储器都是存储数据的，两者有哪些主要区别？

[解析] 合理即可

- 寄存器位于 CPU 中，速度很快，存储容量也较小，存取的代价也较小
- 存储器一般位于 CPU 外，速度稍慢，容量比寄存器大，存取代价较大
- 电路器件的实现原理不同

2.10 什么是微指令？什么是微程序？控制 ROM 的作用是什么？

[解析] 抄书，合理即可

- 微指令：在机器的一个 CPU 周期中，一组实现一定操作功能的微命令的组合，构成一条微指令；微指令=微操作码+执行顺序位。

- 微程序：一条机器指令的功能是由许多条微指令组成的序列来实现的。这个微指令序列通常叫做微程序
- 控制 ROM：指令集中所有指令都对应一段微程序，他们存放在控制 ROM 中，每段微程序的每条微指令都有唯一的地址，当一条指令被执行时，指令译码器对指令操作码进行译码，找到对应微程序的存放地址，开始执行。

[2.13 修改版] 请参照例 2.1，分步骤写出第 3 条加法指令“ADD R1, R0, R1”的执行过程。

[解析] 过程解析可以不严谨，但是取指令和指令执行 2 而阶段必要要有。

- 1) 程序计数器 PC 的内容 0x20000008 送到地址生成部件，寻址内存单元。
- 2) 操作控制器发读信号,将 0x20000008 单元的内容“0x???? ???? ”读出至指令寄存器 IR。
- 3) 程序计数器 PC 自动加 4，指向下一条指令的存放地址 0x2000000C。
- 4) 指令译码器 ID 对指令操作码进行译码，操作控制器 OC 按照操作时序发出相应的控制信号。
- 5) 指令的地址码部分指示操作数在哪。本条指令中的两个源操作数存放在 R0 和 R1，运算结果存放到 R1。操作控制器 OC 控制实现 $R0+R1 \rightarrow R1$ 。

[非教材上的习题]

[补充 2.26] 根据个人理解，阐述计算机抽象分层模型的意义。

[解析] Abstraction Helps Us Manage Complexity(Layers of Representation/Interpretation)

帮助我们理解复杂系统，便于分析计算机各个部件之间的逻辑关系

[补充 2.27] 在按字节编址，采用小端方式的 32 位计算机中，按边界对齐方式为以下 C 语言结构型变量 a 分配存储空间。

```
struct record{
    short x1;
    int x2;
} a;
```

若 a 的首地址为 2024 FE00H，a 的成员变量 x2 的机器数为 1234 0000H，则其中 34H 所在存储单元的地址是（ ）

- A) 2024 FE03H
- B) 2024 FE04H
- C) 2024 FE05H
- D) 2024 FE06H

[解析] D。

- （1）小端模式下低字节对应低地址；
- （2）边界对齐即字对准存放。

C 结构体第一个成员存放的地址为结构体变量偏移量为 0 的地址。其他结构体成员自身对齐时，存放的地址为 min{有效对齐值为自身对齐值，指定对齐值} 的最小整数倍的地址

C 语言中如果没有使用“#pragma pack(n)”指定对齐宽度，默认“指定对齐值”规则是按照结构体中最宽的成员所需要的字节数(本题中 x2 是 4 字节类型，即按照 n=4 对齐存放成员)。

地址	变量	内容
----	----	----

2024 FE00H	x1	
2024 FE01H		
2024 FE02H		
2024 FE03H		
2024 FE04H	x2	00H
2024 FE05H		00H
2024 FE06H		34H
2024 FE07H		12H

[补充 2.28] 2016 年 6 月，由国家并行计算机工程技术研究中心研制的“神威·太湖之光”在全球超级计算机 TOP500 榜单上排名第一。其浮点运算速度为 93.0146 PFLOPS，这说明该计算机每秒完成的浮点数操作次数为（ ）

A) 9.3×10¹³ 次

B) 9.3×10¹⁵ 次

C) 9.3 千万亿次

D) 9.3 亿亿次

[解析] 计算机常用计数单位，实际上这是第 3 章中会讲解的，在第 1 章超级计算机示例中出现过。列为第 2 章的作业有承上启下的意图。

中文单位	中文简称	英文单位	英文简称	十进制（Byte=1）
位	比特	bit	b	0.125
字节	字节	Byte	B	1
开字节	开	KiloByte	KB	10 ³
兆字节	兆	MegaByte	MB	10 ⁶
吉字节	吉	GigaByte	GB	10 ⁹
太字节	太	TeraByte	TB	10 ¹²
拍字节	拍	PetaByte	PB	10 ¹⁵

第 2 章作业 part2

2.12 指令采用固定长度编码有哪些优点？也会带来哪些问题？

[解析]

优点：一次取指操作可以读取一条完整的指令；定长指令执行时间相同，这为采用流水线和超标量等并行处理技术带来了极大便利，有利于性能提升。

缺点：对于复杂的操作，难以在单条指令实现，需要多条指令组合。受指令位数限制，对立即数的大小和格式有限制。

[解析]

WAW 示例: I_{j+1} 试图在指令 I_j 写数据之前写数据，这样最终结果将由 I_j 决定，而程序的本意是保留 I_{j+1} 的结果。

[解析]

(2) 转移代价：假设在指令序列中，指令 I_j 是一条无条件转移指令，其在流水线上的执行步骤为：取指、译码、计算转移地址并更新程序计数器 PC。第三个周期阶数之后，第四个周期将读取转移目标指令 I_k 。但在此之前，指令 I_{j+1} 和 I_{j+2} 也先后进入了流水线，转移必须将这两条指令丢弃。上述过程产生的两个流水线周期延迟被称为转移代价。

(4) **BTB**: 转移目标缓冲器。BTB 收集和存储了近期所有转移指令的有关信息, 并按照查找表的形式组织, 为动态转移预测提供信息。

[解析]

异构多核的目的：兼顾性能和低功耗，以匹配实际应用需求。

- I. 单周期 CPU (指令周期等于 T 周期)
- II. 多周期 CPU (指令周期对应多个 T 周期)
- III. 基本流水线 CPU
- IV. 超标量流水线 CPU

D) 仅 III、IV

多周期 CPU 中 $CPI>1$ ；超标量因为有多条流水线， $CPI<1$ 。

A) 60ps

- [解析] A。
每个流水段时间应取最大的部件时间 80ps，
加上寄存器延时 20ps，
故 CPU 时钟周期是 100ps。选 D。

I. 指令格式规整且长度一致
II. 指令和数据按边界对齐存放
III. 只有 Load/Store 指令才能对操作数进行存储访问

A) 仅 I、II C) 仅 II、III
B) 仅 II、III D) I、II、III

III, 非 Load/Store 体系中, 以加法指令为例, 操作数全部在寄存器需要而时间少于有操作数在存储器的情形, 即指令执行所需的 T 周期不同, 不利于实现流水线

I. I1: add s2, s1, s0 //R[s2] ← R[s1]+R[s0]
 II. I2: load s3, [t2] //R[s3] ← Memory[R[t2]], 加载地址为 R[t2]的存储单元
 III. I3: add s2, s2, s3 //R[s2] ← R[s2]+R[s3]
 IV. I4: store s2, [t2] //Memory[R[t2]] ← R[s2], 保存至地址为 R[t2]的存储单元

A) I1 和 I3
 B) I2 和 I3
 C) I2 和 I4
 D) I3 和 I4

[解析] C。

数据冒险（数据相关），指后一条指令需要用前一条指令的结果

A 选项，I1 中 s2 是结果，但 I3 的源操作数有 s2

B 选项，I2 中 s3 是结果，I2 的源操作数有 s3

C 选项，I2 和 I4 都操作同一个存储单元，资源冲突

D 选项，I4 中 s2 是结果，I3 中源操作数有 s2

[补充 2.33] 改错题。

- （1） 标量处理器中可以利用单条流水线实现整数运算和浮点数运算的并行。
- （2） 流水线机制能够有效提高 CPI (Cycles Per Instruction)和 IPC(Instructions Per Cycle)。
- （3） 相比 CISC，RISC 处理器完成相同功能需要更多的指令，故采用同一个 C 程序在 RISC 上的执行时间长于 CISC 处理器。
- （4） 由于不是标量，向量处理器不能像标量处理器那样使用多条流水线。

[解析]

- （1） 整数运算和浮点数运算规则不同，无法用单一的流水线实现并行。
- （2） 流水线机制能够有效提高 IPC，理想情况下 CPI=1。
- （3） 相比 CISC，RISC 处理器完成相同功能需要更多的机器指令，但 C 程序经编译后的机器代码执行效率可能更高。
- （4） 向量处理器也可以设计多条流水线。

第 2 章作业 part3

2.25 作为一种性能指标，MIPS 是否能客观反映计算机的运算速度？为什么？

[解析]

不同指令集中指令功能有较大差异，单条指令的执行时间不能代表性能。例如，CISC 计算机的单条指令功能在 RISC 计算机上需要多条指令才能实现。

第 3 章作业 part1

3.5 什么是主存储器？什么是外存储器？

[解析]

内存（主存），在主机内部，普遍采用 DRAM，保存计算机运算过程需要的数据。

外存（辅存），掉电后数据不求是，容量大，保存暂时不用的海量数据。

3.6 目前，辅助存储器有哪些常见的接口标准？并简述主要用途。

[解析]

SATA (Serial ATA): SATA 是一种用于连接硬盘驱动器和其他存储设备的接口标准。它是一种串行接口，用于替代传统的并行接口标准，例如 IDE。

SCSI (Small Computer System Interface): SCSI 是一种通用的接口标准，用于连接计算机和其他设备，例如磁盘驱动器、打印机和扫描仪。SCSI 支持高速数据传输和多设备连接。

USB (Universal Serial Bus): USB 是一种通用接口标准，用于连接各种外部设备，包括存储设备、打印机、鼠标和键盘。USB 支持热插拔和高速数据传输。

Thunderbolt: Thunderbolt 是一种高速接口标准，用于连接计算机和其他设备，例如硬盘驱动器、显示器和音频设备。Thunderbolt 支持高达 40Gbps 的数据传输速度。

3.8 EPROM、EEPROM、Flash 的共同特点是什么？这些存储器具有读写功能，为什么仍然是 ROM？

[解析]

共同点：（1）非易失；（2）利用浮栅中是否有电荷来代表“0”或“1”。

非易失存储器被称作 ROM 是历史上形成的习惯称呼。

[非教材上的习题]

[补充 3.31] 以下关于存储器原理描述正确的是（ ）

- I. 硬盘、软盘等磁介质存储器往往依靠磁颗粒的磁化方向来代表了“0”和“1”
 - II. HDD 容量计算公式为：容量=盘片数×磁头数 × 磁道(柱面)数 × 每道扇区数 × 每扇区字节数
 - III. CD-DA、DVD 等光盘上有凹陷和凸起，光照射到凹陷和凸起区域后反射强度不同，代表了“0”和“1”
 - IV. 因为晶体管只有开和关 2 种状态，故而 1 个晶体管只能保存 1 比特信息
- A) I、II
B) II、III
C) III、IV
D) A~C 均不符合题意

[解析] C

III 错误，激光照射到平坦区和凹陷/凸起变化区反射强度不同。

IV 错误，一个晶体管也可以保存多个比特

[补充 3.31] 以下关于计算机外存接口说法正确的是（ ）

- I. IDE (Integrated Drive Electronics) 和 ATA (Advanced Technology Attachment) 都是通用的计算机外设接口标准
- II. SCSI、ATA、SATA 是只能用于硬磁盘 (HDD) 的接口标准
- III. SATA 和 SAS 均采用了串行传输技术
- IV. M.2 NVMe (non-volatile memory express)是目前较为常见的固态硬盘 (SSD) 接口
- V. 嵌入式系统中 UFS 接口的传输速率明显高于 MicroSD 接口

- A) I、II、III
- B) II、III、IV
- C) III、IV、V
- D) I、II、III、IV、V

[解析] C

II 错误，SCSI、ATA、SATA 也可以用作其他类型外设的接口。

[补充 3.32] 以下关于典型 ROM 说法正确的是（ ）

- V. Mask ROM（掩模 ROM）中的数据用户不能修改
- VI. ROM 芯片中字线用来选中存储单元，而位线用于数据的输入和输出
- VII. PROM (Programmable ROM)可以利用外围电路实现数据的多次写入
- VIII. EPROM (Erasable PROM)可以利用外围电路实现数据的多次修改
- IX. EEPROM (Electrically EPROM)可以利用外围电路实现数据的多次修改
- X. Flash 和 E²PROM 都基于浮置栅中是否存有电荷来实现“0”或“1”的逻辑状态存储
- XI. NAND Flash 比 NOR Flash 存储密度更高是因为少一个选通管

- A) I、III
- B) II、III
- C) III、IV
- D) V、VI

删除[陈晓辉 [2]]: II、
删除[陈晓辉 [2]]: 、IV
删除[陈晓辉 [2]]: 、V
删除[陈晓辉 [2]]: 、VII

[解析] D

III 错误，PROM (Programmable ROM)只能一次写入

IV 错误，EPROM (Erasable PROM)擦除需要使用紫外线照射

VII 错误，NAND Flash 比 NOR Flash 存储密度更高主要是因为位线采用串联而非并联的方式

[补充 3.33] 以下关于计算机存储器说法正确的是（ ）

- I. 从物理机制上看，任何一种存储器都要能表示“0”和“1”两种状态
- II. 从物理机制上看，无论是磁存储、光存储还是半导体存储，都只能表示表示“0”和“1”两种状态
- III. 硬磁盘（HDD）的容量≈盘片数×磁头数 × 磁道(柱面)数 × 每道扇区数 × 每扇区字节数
- IV. 光盘的容量往往受限于激光头组件的波长
- V. 半导体存储器往往受限于芯片面积而难以实现超大容量
- VI. 半导体存储与磁存储、光存储相比，具有体积小、重量轻的优点
- VII. 半导体存储被广泛用于芯片内存储、计算机主存和外存

- A) I、II、III、IV
- B) II、III、IV、V
- C) I、II、III、V
- D) IV、V、VI、VII

[解析] D

II 错误，磁存储、光存储或者半导体存储都有可能存储多个状态，只是存储 2 个状态技术实现较为容易。

[补充 3.34] 以 Intel 2164A 为例，定性分析 DRAM 芯片写操作过程中哪些步骤会产生不可忽略的延时（Latency）。

[解析] 应分析从发送地址开始到存储器输出数据到数据总线上的全过程。包括：行地址发送、列地址发送、地址译码；选中的存储单元输出数据稳定需要时间，等。

【拓展思考】弄清楚延时的来源，对于理解存储器快速访问技术很有帮助，如“发送行地址—发送列地址—读写数据”3 个阶段在访问连续数据时，可以简化为“发送行地址—读写数据”2 个阶段。再如，SDRAM 芯片普遍支持的“突发总线模式（Burst）”。

【拓展思考】这种分阶段操作，和“流水线”思想能否结合？后续第 4 章总线中以 AMBA 总线为例，还会探讨提升访存效率的思路。

[补充 3.35*选做] 某品牌的内存芯片标称了“DDR5-3200 22-22-22”，已知芯片共有 64 根数据线，下列说法正确的是（ ）

- I. 该芯片的工作时钟是 1600 MHz、时钟周期是 0.625ns
 - II. 该芯片的理论最高数据传输速率是 $3200\text{ MT/s} \times 64\text{ bits} = 25.60\text{ GB}$
 - III. “tCL-tRCD-tRP”（本题中“22-22-22”）数值对应的是时钟周期的个数
 - IV. “tCL-tRCD-tRP”（本题中“22-22-22”）数值对应的是 ns
 - V. 该芯片的 tCL (CAS Latency) 是 13.75 ns
 - VI. 该芯片的理论最高数据传输速率是 $1 / (13.75\text{ ns}) \times 64\text{ bits} \approx 0.58\text{ GB}$
- A) I、II、III、IV
B) I、II、IV、V
C) I、II、III、V
D) I、II、III、VI

[解析]

IV 不对，“tCL-tRCD-tRP”按照时钟周期的个数计量。

VI 不对，Latency 和指访问数据的延时，和数据传输速率不同。

【拓展思考】要注意区分不同术语定义对应的物理意义，类似易混淆的术语还有计算机网络中将会学习的吞吐量和延时（有些通信方式延时很大，但是吞吐量很高）。

[补充 3.36*选做] 对比分析随机读/写典型硬磁盘（HDD）和典型 NAND 型固态硬盘（SSD）过程中延迟的产生原因。

[解析]

HDD 延迟

随机读取：随机寻道延迟（约 4~5ms），磁→电，数据至硬盘内 Cache，硬盘对外 I/O 接口。

NAND Flash：仅仅是电信号的产生的延迟（很小），没有机械控制的延迟。

//每个 Bit Line 下的基本存储单元是串联的，NAND 是以 Page 为读取单位和写入单位，以 Block 为擦除单位。

[补充 3.37*选做] 通过搜索引擎或 DeepSeek 调研国产存储器主流厂家。下列关于国产存储器芯片说法正确的是（ ）

- I. 长鑫存储创立于 2016 年，同年长鑫存储（合肥）DRAM 基地一期项目诞生
- II. 成立于 2008 年的合肥京东方，主要产品是 DRAM 芯片
- III. 长鑫存储在 2023 年推出 LPDDR5 芯片
- IV. 长鑫存储于 2024 年底实现 DDR5 内存芯片的量产
- V. 成立于 2016 年的长江存储，主要产品是 3D NAND 闪存
- VI. 嘉合劲威（POWEV），主要产品是 DDR4 内存条
- A) I、II、III、IV
- B) II、III、IV、V
- C) I、III、IV、V
- D) II、III、V、VI

[解析] C

II. 成立于 2008 年的合肥京东方，主要产品是 DRAM 芯片
京东方是生产显示屏的

待定内容

[补充 3.36*选做] ???。
通过 <https://www.lcsc.com/> 查询长鑫存储内存产品的手册（关键词“CXMT”），
阅读“CXMT CXDQ3A8AM-CJ-A”的数据手册
可 以 直 接 访 问
https://www.lcsc.com/datasheet/lcsc_datasheet_2410121531_CXMT-CXDQ3A8AM-CJ-A_C20598569.pdf

根据课程所学知识，分析手册中下表所列各个信息项含义。

Speed	DDR4-3200	Unit
	22-22-22	
tCK (min)	0.625	ns
CAS Latency	22	nCK
tRCD (min)	13.75	ns
tRP (min)	13.75	ns
tRAS (min)	32	ns
tRC (min)	45.75	ns

根据课程所学知识，分析手册中“Table 1-2”中各个信息项含义。

Table 1-2 8Gb Addressing Table

Parameter	1G x 8
Number of Bank Groups	4
Number of Banks per Bank Group	4
Bank Group Address	BG0~BG1
Bank Address per Bank Group	BA0~BA1
Row Address	A0~A15
Column Address	A0~A9
Page Size	1KB

<https://library.techinsights.com/public/hg-asset/97c38cd6-27c0-489d-8bec-c7ac6ac0ac4b>

200b x32 Mobile LPDDR4X SDRAM Datasheet

https://www.lcsc.com/datasheet/lcsc_datasheet_2410121605_CXMT-CXDB4CBAM-MK-A_C20598565.pdf

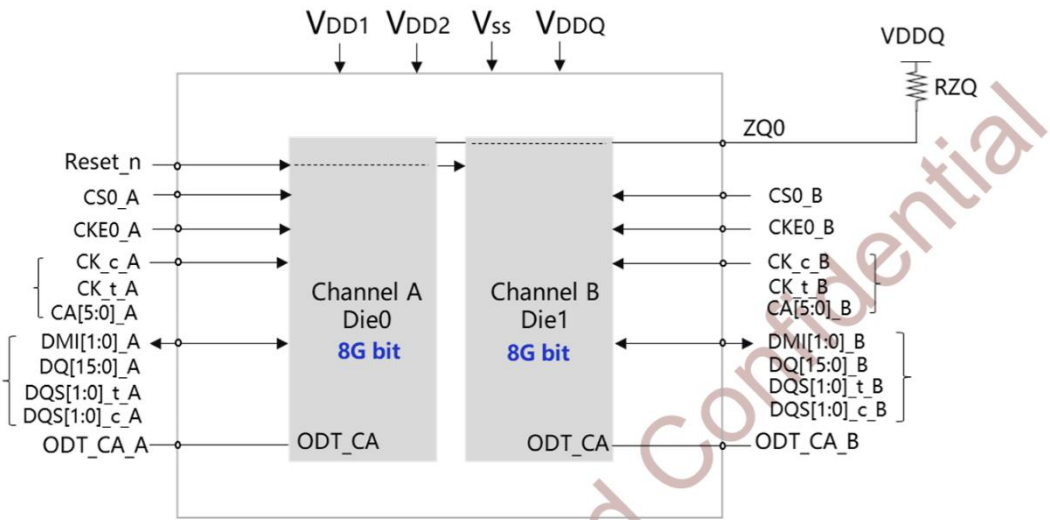


Figure 2-3 Dual-Die, Dual-Channel, Single-Rank Package Block Diagram (2GB x32 I/O)

第 3 章作业 part2

3.22 某按字节编址的计算机系统，使用了 40 位地址线，16 位数据线，请问该计算机存储器空间的最大寻址范围是什么？

[解析]

0x00 0000 0000 ~ 0xFF FFFF FFFF

[补充 3.38] 某按字节编址的计算机系统，有 14 根地址线，8 根数据线，用 4 片 4K×8 位的 SRAM 芯片构成 16K×8 位的存储器。请①画出 4 片 SRAM 芯片与 CPU 的连接图；并②列出 4 片 SRAM 芯片的地址范围。

[解析]

等于 CPU 而言，可以产生 0~16K 的地址范围。

4 片 SRAM 芯片的数据线与 CPU 数据线连接（并联）；低 12 根地址线与 CPU 地址线低 12 位连接（并联）。

芯片 1 地址线 A[13]、A[12]均经非门与 CPU 的 A[13]、A[12]连接

芯片 2 地址线 A[13]经非门与 CPU 的 A[13]连接，芯片 2 的 A[12]直接连接 CPU 的 A[12]

芯片 3 地址线 A[13]直接连接 CPU 的 A[13]，芯片 3 的 A[12]经非门连接 CPU 的 A[12]

芯片 4 高 2 位地址线 A[13]、A[12]均直接连接 CPU 的 A[13]、A[12]

芯片 1 地址范围：0x0000 ~ 0x0FFF

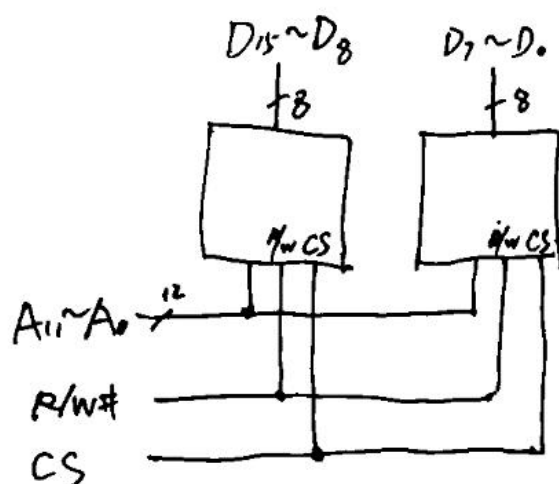
芯片 2 地址范围：0x1000 ~ 0x1FFF

芯片 3 地址范围：0x2000 ~ 0x2FFF

芯片 4 地址范围：0x3000 ~ 0x3FFF

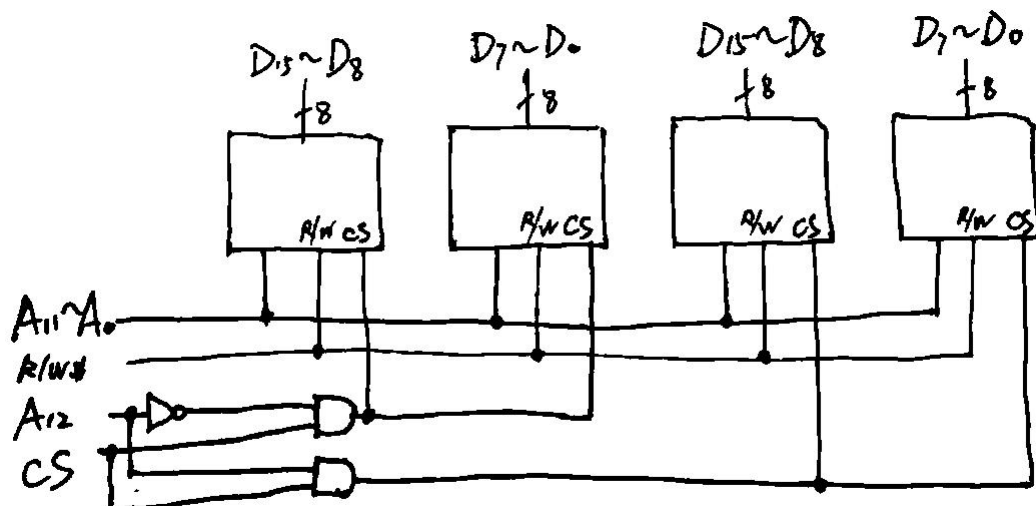
3.24 试用 4K×8 位的芯片构成 4K×16 位的存储器。

[解析]



3.27 设计一个用 4 片 4K×8 位的芯片构成 8K×16 位的存储器。

[解析] 答案不唯一，合理即可



[修改版]3.10 从 DDR 到 DDR5 有哪些改进？

[解析]

SDR SDRAM：采用同步方式，支持 Burst 模式

DDR SDRAM：在时钟上沿和下沿都进行一次写或读操作

DDR2 SDRAM：4bits 预读取

DDR3 SDRAM：8bits 预读取

DDR4 SDRAM：8bits 预读取/Bank Group、点到点传输、3D 堆叠

DDR5 SDRAM：16bits 预读取/Bank Group、容量最高可达 512GB

3.18 Cache line 应该含有那些基本信息项？

[解析] 1.数据字段 2.标记字段 3.有效位字段 4.一致性控制位字段 5.替换控制位字段

3.19 Cache 常用的替换策略有哪些？

[解析]

1.随机替换策略 2.最不经常使用替换策略 3.先进先出替换策略 4.近期最少使用替换策略

[补充 3.39] 以下关于 Cache 说法正确的是（ ）

- A) 贯穿读出（Look Through）方式下，CPU 得到的所有数据都是从 Cache 中读出的
- B) 旁路读出（Look Aside）方式下，CPU 得到的所有数据都是从 Cache 中读出的
- C) 写通方式（Write Through）方式下，CPU 写入存储器的数据可能不经过 Cache
- D) 写回方式（Write Back）方式下，CPU 写入存储器的数据可能不经过 Cache

[解析] C

A、B，如果数据不在 Cache 中，CPU 访问主存，同时将数据（所在的主存数据块）装入 Cache，并设置 Cache 块有效位字段。

D 写回方式（Write Back）方式下，更新数据只写到 Cache

3.20 某计算机按字节编址，其主存容量为 1MB，Cache 容量为 16KB，Cache 和主存之

间交换的块大小为 64B，采用直接相联映射方式。

(1) Cache 共有多少个字块 (Cache line) ?

(2) 主存地址为 02021H 的单元装入 Cache 后对应的 Cache 地址是?

(3) 主存地址为 02021H 的单元装入 Cache 后存放在 Cache 中的第几字块中 (Cache 起始字块为第 0 字块) ?

[解析]

(1) 字块数=Cache 容量/块大小= $2^{14}/2^6=2^8$

(2) 直接相联映射方式的主存地址由三部分组成: 页号T, 块号C, 块内地址W

其中, $T=\text{主存容量}/\text{Cache 容量}=2^{20}/2^{14}=2^6$, 故T 有6 位; 由(1)知, C 有8 位;

因为块大小=64B= 2^6 , 故W 有6 位。 由此可对主存地址02021H 进行分析:

00000010000000100001(T:蓝色, C:绿色, W:红色)

由直接相联映射, 该单元装入cache 的第128(10000000)块, 块内地址为100001。因此得cache 地址100000000100001, 即2021H

(3)由(2)知该单元装入cache 的第128(10000000)块。

3.21 某计算机按字节编址, 其主存容量为 1MB, Cache 容量为 16KB, Cache 和主存之间交换的块大小为 64B, 采用 8 路组相联映射方式。

(1) 主存地址中页号 s、页内块号 u、块内地址 W 各占多少位?

(2)主存地址为 02021H 的单元装入 Cache 后存放在 Cache 中的第几组(起始组为第 0 组)?

(3) Cache line 对应的 Tag 字段占用多少位?

[解析]

(1)组相联映射方式的主存地址由三部分组成: 页号s, 块号u, 块内地址W

页内块号u=cache 中组号=cache 容量/(组中数据块数目*数据块大小)= $2^{14}/2^6*2^3=2^5$, u 有5 位。

因为块大小=64B= 2^6 , 故W 有6 位。

页号s 位数=20-6-5=9.

(2)对主存地址02021H 进行分析:

00000010000000100001(s:蓝色, u:绿色, W:红色)

故存放在第0(00000)组

(3)Tag 字段位数=页数位数=9

3.12 虚拟存储器解决了哪些问题?

[解析] 要点合理即可

内存不足: 虚拟存储器允许计算机使用硬盘上的存储空间作为虚拟内存。

效率: 将用到的进程/程序的一部分加载到物理内存中, 而不是整体加载。

编程的灵活性: 程序员在虚拟地址空间编程。

安全性: OS 将不同进程的虚拟地址空间映射到不同的物理地址空间, 形成隔离。

[补充 3.40] 假定主存地址为 32 位, 按字节编址, 指令 Cache 和数据 Cache 与主存之间均采用 8 路组相联映射方式、直写(Write Through)写策略和 LRU 替换算法, 主存块大小为 64B, 指令 Cache 和数据 Cache 容量均为 32KB, 开始时 Cache 均为空。请回答下列向题。

(1) Cache 每一行中标记(Tag)、LRU 位各占几位? 是否有修改位?

(2) 有如下 C 语言程序段:

```
for(k=0; k<1024; k++) s[k]=2*s[k];
```

若数组 s 及变量 k 均为 int 型，int 型数据占 4B，变量 k 分配在寄存器中，数组 s 在主存中的起始地址为 0080 00C0H，则该程序段执行过程中，访问数组 s 的数据 Cache 缺失次数为多少？

[解析]

(1)主存块大小为 64B=2⁶字节,故主存地址低 6 位为块内地址,Cache 组数为 32KB/(64B×8)=64=2⁶,故主存地址中间 6 位为 Cache 组号，主存地址中高 32-6-6=20 位为标记，采用 8 路组相联映射，故每行中 LRU 位占 3 位，采用直写方式，故没有修改位。

(2)因为数组 s 的起始地址最后 6 位全为 0，故 s 位于一个主存块开始处，占 1024×4B/64B=64 个主存块：执行程序段过程中，每个主存块中的 64B/4B=16 个数组元素依次读、写 1 次，因而对于每个主存块，总是第一次访问缺失，以后每次命中。综上，数组 s 的数据 Cache 访问缺失次数为 64 次。

【拓展思考】对于追求高性能的程序，程序中所需处理的数据如果能全部载入 Cache，代码运行效率将有极大的提升。

[补充 3.41] 某计算机主存地址为 24 位,采用分页虚拟存储管理方式,虚存空间大小为 4GB,页大小为 4KB，按字节编址。某进程的页表部分内容如下表所示。

虚页号	实页号（页框号）	存在位
82	024H	0
...	...	...
129	180H	1
130	018H	1

- 当 CPU 访问虚拟地址 0008 2840H 时，虚实地址转换的结果是。（ ）
- A) 得到主存地址 02 4840H
 - B) 得到主存地址 18 0840H
 - C) 得到主存地址 01 8840H
 - D) 检测到缺页异常

[解析] C

页大小为 4KB → 地址地 12 位是页内偏移，即 840H；

虚页号是高 12 位，082H → 130D；

虚页号 130D 在页表中存在 → 查表得到 018H → 实地址是 018840H。

此题有个细节易被忽略，页表中虚页号是十进制表示，实页号是 16 进制表示。

第 4 章作业 part1

4.2 举例说明何为总线复用？

[解析]

一条或一套物理信号线在不同时间用作不同功能。

例如，第 3 章 DRAM 芯片 Intel 2164A，行地址 A7~A0 和列地址 A15~A8，分时复用。

再如，8086 的 16 根数据线和低 16 位的地址总线复用。

[修改版] 4.5 请给出典型的 PC 总线示意图和典型基于 AMBA2 的 MCU 系统的总线示意图。

[解析] 教材 p152 图 4.7；教材 p161 图 4.18

4.8 计算机系统什么情况下需要总线仲裁（arbitration）？

[解析]

当多个主设备同时申请总线时，需要判定哪个主设备能优先使用总线，这时需要总线仲裁。

4.13 异步总线有哪些可能的握手方式？

[解析]

不互锁方式 半互锁方式 全互锁方式

4.15 周期分裂式总线操作时序有哪些特点？适用于什么样的场景？

[解析]

特点：将一个总线读周期或写周期分解成两个分离的子周期：寻址子周期，数据传送子周期。

适用场景：多主模块系统，从模块不能随时及时响应。

4.19 为什么 AMBA 总线中没有定义电气特性和机械特性？

[解析]

AMBA 是片内总线，是一种与工艺无关的片上协议

4.21 简述 AHB 总线的流水线机制。

[解析]

本次数据传输操作的地址阶段和上次操作的数据阶段重复。

[修改版] 4.22 定性分析 AHB 中 SPLIT 操作的优点。

[解析]

从机不能响应时，可以把总线的使用权让给其他主机。这样做支持多个主机同时访问 AHB 总线，降低总线冲突的发生。

4.23 解释图 4.21 中 HREADY 信号的作用。

[解析]

HREADY 用于插入等待周期，当 HREADY 为低电平时，数据和地址的传输暂停，为从机准备数据提供时间，恢复高电平后传输继续。

4.25 AHB 中突发传输定义了哪些类型？各自有什么特点？

[解析]

- 单次传输，每次传输单个数据
- 不确定长度传输，地址顺序递增
- 指定长度的地址顺序递增式传输

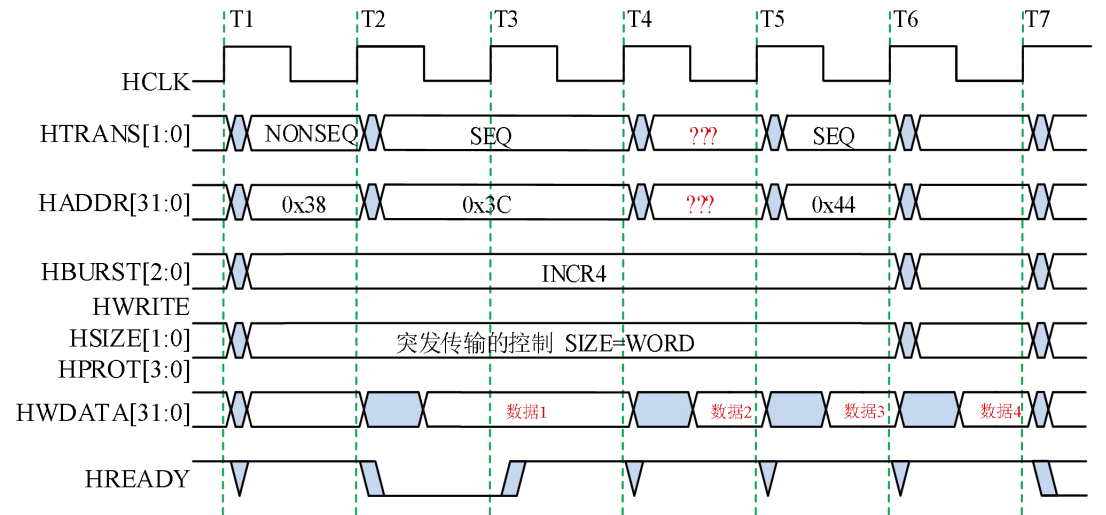
指定长度的地址循环递增式传输

4.30 PCIe 5.0 版本中 X16 的吞吐量 63.0 GB/s 是如何计算得到的？

[解析]

$32.0\text{ GT/s} * 16 * (128\text{b}/130\text{b}) / 8$

[补充 4.62] 下图中 HTRANS[1:0]信号“???”应该为什么取值？HADDR[31:0]信号“???”应该为什么取值？HWDATA[31:0]中数据 1、数据 2、数据 3、数据 4 对应的地址分别是什么？



[解析]

- 单次传输，每次传输单个数据
- HTRANS[1:0]信号“???”是 SEQ
- HADDR[31:0]信号“???”是 0x40
- HWDATA[31:0]中数据 1 数据 2、数据 3、数据 4 对应的地址分别是：0x38、0x3C、0x40、0x44

[补充 4.63] QPI 总线是一种点对点全工同步串行总线，总线上的设备可同时接收和发送信息，每个方向可同时传输 20 位信息（16 位数据+4 位校验位），每个 QPI 数据包有 80 位信息，分 2 个时钟周期传送，每个时钟周期传递 2 次，因此，QPI 总线带宽为：每秒传送次数×2B×2. 若 QPI 时钟频率为 2.4GHz,则总线带宽为（ ）

- A) 4.8 GB/s
- B) 9.6 GB/s
- C) 19.2 GB/s
- D) 38.4 GB/s

[解析] C

每个时钟周期传递 2 次→每秒传输次数 2.4GHz ×2 = 4.8G
根据题意，带宽=每秒传送次数×2B×2=4.8G×2B×2=19.2 GB/s

第 4 章作业 part2

4.35 异步串行通信中的起始位和停止位有什么作用？

[解析]

标识传输的开始和结束。

4.41 CPU 与 I/O 设备之间的数据传送有哪几种方式？每种工作方式的特点是什么？

[解析]无条件传送方式、状态查询传送方式、中断传送方式和 DMA 传送方式。

- 无条件传送方式：外设工作时始终处于“准备好”的状态，CPU 对无条件外设进行输入输出操作时不需要考虑其状态。
- 状态查询传送方式：CPU 访问数据端口前，先查询该设备的状态；设备“准备好”时，CPU 才对数据端口进行输入/输出。
- 中断传送方式：在中断传输方式中，平时 CPU 执行正常的处理任务，当外设状态改变或者需要与 CPU 进行数据交换时，由 I/O 接口主动向 CPU 发出数据传送请求，CPU 中断当前的任务执行，转而执行相应的中断服务程序，为外设（I/O 接口）进行服务。
- DMA 传送方式：在内存与 I/O 接口之间直接进行传输，无需 CPU 干预。适合大数据量的快速传送。

4.46 数据块传送方式的 DMA 适用于什么场景？

[解析]

适合大数据量的快速传送。

4.58 异步串行通信系统中，采样数据时为什么要在数据位的中间？

[解析]

提高采样的准确率，避免因为干扰而误采样，靠近传送波形的上升沿或者下降沿都有可能采样到相邻位的信号。

4.61 描述 I²C 总线协议中的状态，并画出状态转移图。

[解析]

总线空闲 I：SDA 与 SCL 均为高电平

启动数据传输 S：SCL 为高电平，SDA 负跳变

停止数据传输 P：SCL 为高电平，SDA 正跳变

重新启动 Sr：SCL 为高电平，SDA 负跳变

数据有效 D：SCL 为高电平期间，SDA 电平稳定

等待/ 数据无效 Q：SCL 保持低电平，此时可修改线上数据

应答 A /不应答 N：接收方将 SDA 拉低发出 ACK 或释放 SDA 发出 NACK

VI，错误，I/O 端口地址取决于地址译码电路的设计，可以在任意地址段

[补充 4.66] 以下关于 SPI 总线说法正确的是（ ）

- I. SPI 是一种全双工通信总线，即主设备读从设备、主设备写从设备可同时进行
- II. 采用 SPI 通信时，主设备和从设备是共用时钟信号的
- III. SPI 主设备通过 CS# 信号指示从设备数据传输的开始与结束
- IV. SPI 是同步通信总线
- V. SPI 总线是串行通信总线
- VI. 从设备向主设备发送数据既可以在时钟上升沿，也可以在时钟的下降沿
- VII. 接收设备采样数据线的总是滞后发送设备推送数据半个周期

- A) 仅 I、II、III、IV
- B) 仅 II、III、IV、V
- C) 仅 III、IV、V、VI
- D) 仅 IV、V、VI、VII
- E) 以上答案均不正确

[解析] E。

所有说法都是正确的

第 5 章作业

-----PART1-----

5.1 计算机中的“ISA”和“ μ arch”各是什么意思？两者之间有何联系？

[解析]

- ISA: 指令集体系结构，用来描述软件如何使用硬件的一种规范和约定
- μ arch: 微架构，对应于 ISA 的硬件实现方式
- 相同的 ISA 的处理器可以有不同微架构，ISA 是介于微架构硬件实现和软件的桥梁

5.2 请简述哈佛结构的主要优缺点。

[解析]

- 优点: 取指和数据存取可以通过两套独立的总线同时进行，从而减小指令流水线发生资源冲突的概率
- 缺点: 哈佛结构较为复杂，与外设以及扩展存储器的连接难度较大

5.3 TCM 与高速缓存 Cache 有什么区别？

[解析]

TCM 具有物理地址，需要占用内存地址空间；Cache 是透明的。

5.6 MMU 和 MPU 的功能有何异同？

[解析]

- 二者都服务于存储器管理。
- **MMU**：内存分页管理+分区域访问权限管理+虚拟地址 **VA** 到物理地址 **PA** 的转换，适用于多用户系统。
- **MPU**：内存分区域访问权限管理。

5.11 Cortex-M3/M4 的 CODE 区选用总线互连矩阵与总线复用器有什么区别？

[解析]

复用只能分时使用总线。

总线互联矩阵由于存在内部交换开关矩阵，可允许并行传输发生。

5.14 I-Code 和 D-Code 总线全部连接到同一片 Flash 芯片上会有什么问题？

[解析]

导致 **I-Code** 和 **D-Code** 无法做到并行操作，降低效率。

5.15 私有外设总线（Private Peripheral Bus，PPB）基于哪种总线协议，有何特点？

[解析]

32 位 APB 总线。

负责连接特定的地址范围：外部私有外设子区域 **0xE0040000~0xE00F FFFF**。

[补充 5.43] *寒武纪芯片在实现 AI 算法加速时，针对向量和矩阵操作指令，使用片内的 **Scratchpad Memory** 替代传统的寄存器。自行阅读寒武纪芯片相关资料（如下为原始论文），简析 CPU 内的 Cache、CPU 内的 **Scratchpad Memory** 和 CPU 内的 Cache 三者有何区别？

S. Liu et al., "Cambricon: An Instruction Set Architecture for Neural Networks," 2016 ACM/IEEE 43rd Annual International Symposium on Computer Architecture (ISCA), Seoul, Korea (South), 2016, pp. 393-405, doi: 10.1109/ISCA.2016.42.

<https://ieeexplore.ieee.org/document/7551409>

[解析]

- **Scratchpad Memory** 本质上是课程中讲得 **TCM**，**TCM** 具有物理地址，需要占用内存地址空间；**Cache** 是透明的。**SPM (Scratchpad Memory)** 和主存统一编址，处理器可以直接访问 **SPM** 中的数据。
- 寄存器数量太少，不适合向量和矩阵计算。**Scratchpad Memory** 容量比寄存器组大很多。**Scratchpad Memory** 寻址方式和寄存器不同（**Load**、**Store** 指令实现）。

[补充 5.44] *自行查阅资料，**Lunar Lake Architecture** 和 **Raptor Lake Architecture** 这两种微架构各自用于 Intel 的什么产品系列？

[解析]

Lunar Lake Architecture 用于 **Core Ultra**

Raptor Lake Architecture 用于 **13/14th Core**

[补充 5.45] *从微架构角度，简析 Intel 公司 **Lunar Lake Architecture** 和 **Raptor Lake**

Architecture 最大的差异是什么。

[解析]

Lunar Lake Architecture 集成了 NPU、GPU 核心能力更强。

<https://www.hwcooling.net/en/intel-lunar-lake-the-most-efficient-x86-processor-detailed/>

The NPU of Lunar Lake processors features a new architecture (NPU 4) compared to that of Meteor Lake processors. NPU 4 contains 3× more matrix blocks (12,000 in 6 NCE blocks), has 12× more powerful SHAVE vector units than the NPU 3 in Meteor Lake, and supports INT8 (2048 MAC operations per cycle) and FP16 operations (at half-performance compared to INT8 though, 1024 FMAC operations per cycle). This architecture is claimed to achieve 2x better power efficiency.

[补充 5.46] *查阅如下资料，分析课程中 ARM 公司的什么技术与 “Intel 从第 12 代酷睿处理器开始采用的 Hybrid Architecture”是同类技术。

<https://www.intel.cn/content/www/cn/zh/developer/articles/technical/hybrid-architecture.html>

[解析] 异构多核，ARM 公司从 big.LITTLE → DynamIQ

-----PART2-----

5.10 Cortex-M3 与 Cortex-M4 使用两个堆栈的目的是什么？在中断响应时，程序断点和程序状态寄存器的内容保存在哪个堆栈中？

[解析]

- 使用两个堆栈是为了服务于不同的操作模式和特权访问等级，处理模式总是使用 MSP，线程模式可以使用 MSP 或 PSP。
- 程序断点和程序状态寄存器的内容保存在 MSP 中。

5.13 Cortex-M3/M4 从 SRAM 域读取指令执行时有什么缺点？

[解析]

从 CODE 区域利用 I-CODE 读取指令不会和从 SRAM 读取数据冲突，可并行。

5.17 在 Cortex-M3/M4 中，寄存器 R0~R12 有何异同？如果这些寄存器都是空闲的，你觉得首先使用哪些？为什么？

[解析]

- 相同点：都是通用寄存器
- 不同点：为了能够实现汇编程序与 C 语言程序的相互调用，ARM 公司制定了 AAPCS (ARM Architecture Procedure Call Standard) 规范：R0~R3 用于子程序之间的参数传递；R4~R11 用于保存子程序的局部变量；R12 作为子程序调用中间寄存器。
- 优先使用低位寄存器
 - ☐ R0~R7 低位寄存器(许多 16 位的 thumb 指令只能访问低位寄存器)
 - ☐ R8~R12 高位寄存器(可用于 32 位指令和少数几个 16 位指令)

5.19 某段程序需要跳转到 0x0100 0000 执行，有人写了如下两行汇编指令代码：

```
MOV    R0,    #0x0100 0000
MOV    R15,   R0
```

请问这样会有什么问题？

[解析]

无论使用跳转指令还是直接写 PC 寄存器，写入值必须是奇数，确保其最低位是“1”，以表示其处于 Thumb 状态，否则将被认为试图转入 ARM 模式，从而导致出现错误异常。

[补充 5.47] 以下关于 Cortex-M3/M4 操作状态与操作模式法正确的是（ ）

- VIII. 特权线程模式下可以通过置位 CONTROL 寄存器 nPRIV 位进入非特权模式
 - IX. 非特权线程模式下可以通过置零 CONTROL 寄存器 nPRIV 位进入特权模式
 - X. 处理模式下可以通过置零 CONTROL 寄存器 nPRIV 位进入特权线程模式
 - XI. 非特权模式下访问 CONTROL 寄存器会触发 Usage Fault 异常
 - XII. 非特权线程模式下访问存储器可能会触发 MemManage Fault 异常
 - XIII. 处理模式下可以通过置零 CONTROL 寄存器 nPRIV 位进入特权线程模式
 - XIV. 处理模式下只能使用 MSP 而不能使用 PSP
 - XV. 线程模式下只能使用 PSP 而不能使用 MSP
- E) I、II、III、IV、V
F) II、III、IV、V、VI
G) III、IV、V、VI、VII
H) IV、V、VI、VII、VIII
I) 以上选项均不正确

[解析] C

- I. 特权线程模式下可以通过置位 CONTROL 寄存器 nPRIV 位进入非特权模式
- II. 非特权线程模式下不能访问 CONTROL 寄存器
- III. 处理模式下可以通过置零 CONTROL 寄存器 nPRIV 位进入特权线程模式
- IV. 非特权模式下访问 CONTROL 寄存器会触发 Usage Fault 异常
- V. 非特权线程模式下访问存储器可能会触发 MemManage Fault 异常
- VI. 处理模式下可以通过置零 CONTROL 寄存器 nPRIV 位进入特权线程模式
- VII. 处理模式下只能使用 MSP 而不能使用 PSP
- VIII. 线程模式下既可以使用 PSP 也可以使用 MSP

[补充 5.48] Cortex-M3/M4 系统中，栈存储区域放置在 4GB 地址空间的什么位置比较好？请解释原因。

[解析]SRAM 最低的地址范围，因为 Cortex-M3/M4 只支持满递减堆栈，放置在这个区域，如果发生栈溢出，不会影响到正常的数据。

[补充 5.49] 阅读教材 7.4.1（p407~p408），简述 ARM 公司 AAPCS 的基本内容。

[补充 5.50] *x86 架构中，函数参数传递的约定（calling conventions）规定了函数如何接收参数以及如何返回值。阅读相关资料（如下），简析 __cdecl() 和 __fastcall() 两种调用方式的差异。

Argument Passing and Naming Conventions | Microsoft Learn
<https://learn.microsoft.com/en-us/cpp/cpp/argument-passing-and-naming-conventions?view=msvc-170>
自变量传递和命名约定 | Microsoft Learn

<https://learn.microsoft.com/zh-cn/cpp/cpp/argument-passing-and-naming-conventions?view=msvc-170>

Keyword	Stack cleanup	Parameter passing
__cdecl	Caller	Pushes parameters on the stack, in reverse order (right to left)
__clrcall	n/a	Load parameters onto CLR expression stack in order (left to right).
__stdcall	Callee	Pushes parameters on the stack, in reverse order (right to left)
__fastcall	Callee	Stored in registers, then pushed on stack
__thiscall	Callee	Pushed on stack; this pointer stored in ECX
__vectorcall	Callee	Stored in registers, then pushed on stack in reverse order (right to left)

[解析]

[__cdecl\(\)](#)调用方式

所有参数通过堆栈传递，Right to left.

[__fastcall\(\)](#) 两种调用方式

The first two DWORD or smaller arguments that are found in the argument list from left to right are passed in ECX and EDX registers; all other arguments are passed on the stack from right to left.

[补充 5.51] *阅读相关资料（如下），简析 RISC-V calling convention 中函数参数为 int 类型情形，和参数为 double 类型情形有何不同。

<https://riscv.org/wp-content/uploads/2024/12/riscv-calling.pdf>

[解析]

[补充 5.52] **ABI（Application Binary Interface）：应用程序二进制接口，描述了应用程序和操作系统之间，一个应用和它的库之间，或者应用的组成部分之间的接口。查阅 Windows ABI 资料，了解 ABI 主要约定的内容包括哪些？

x64 ABI conventions | Microsoft Learn

<https://learn.microsoft.com/en-us/cpp/build/x64-software-conventions?view=msvc-170>

-----PART3-----

5.16 如果非特权线程试图访问内核私有区域，将会导致哪一类异常？如果 Cortex-M3 使用了一条 SIMD 运算指令，结果又将如何？

[解析]

- 访问内存的行为违反了 MPU 定义的规则，触发 MemManage 错误
- 由于 cortex-M3 没有浮点运算，DSP 等协处理器，访问位配置的部件，触发用法错误

5.22 某基于 Cortex-M4 的 SOC 芯片共有 64 级外部中断，BASEPRI 寄存器的宽度共有几位？
如果想屏蔽所有优先级大于 16 的中断，请写出对 BASEPRI 寄存器进行设置的汇编指令。如
果想屏蔽所有优先级大于 0 的中断，又该如何设置？

[解析]

由题知芯片配置了 $64=2^6$ 级外部中断，故 BASEPRI:7:2 共 6 位

屏蔽所有优先级大于 16 的中断：

```
MOV R0, #0x0000 0044 或者 MOV R0, #0b 010001(00)
```

```
MSR BASEPRI, R0
```

屏蔽所有优先级大于 0 的中断：

```
MOV R0, #0x 0000 0004 或者 MOV R0, #0b 000001(00)
```

```
MSR BASEPRI, R0
```

5.23 有人写了一段对 Cortex-M4 的进程栈进行初始化的代码，其中 PSP 的初始值设为 0x8765 4321，并且使用了如下一条语句：“MOV PSP, R0”对 PSP 进行赋值（其中 R0=0x8765 4321）。
这样做存在哪些问题？请逐一说明。

[解析]

- 由于堆栈操作是以字为单位的，所以堆栈要字对齐，而 PSP 的初始值并没有做到字对齐
- MSP 和 PSP 只能用特殊寄存器指令 MRS、MSR 指令访问

5.29 Cortex-M3 存储空间的哪些区域支持位段（bit-band）操作？

[解析]

SRAM 区域的最低 1MB（0x20000000 ~ 0x200FFFFF）

外设区域的最低 1MB（0x40000000 ~ 0x400FFFFF）

5.31 写出利用位段操作读取 0x4000 1000 的第 3 位的代码。

[解析] 此处第 3 位的理解可能有歧义，一种是理解为 bit[2]、一种是理解为 bit[3]，以下按照 bit[3]给出参考代码

汇编实现：

```
LDR R0, =0x4202 000C
```

```
LDR R1, [R0]
```

C 实现（volatile 关键字非必须）：

```
int x;
```

```
x = *((volatile unsigned long*) (0x4202 000C));
```

5.32 存储器访问属性包括哪些？

[解析]

- 可缓冲：当处理器继续执行下一条指令时，对存储器的写操作可以由写缓冲执行
- 可缓存：读存储器所得到的数据可被复制到缓存，下次再访问时可以从缓存中取出这个数值，从而加快程序执行

- 可执行：所得到的数据可被复制到缓存，下次再访问时可以从缓存中取出这个数值从而加快程序执行
- 可共享：这种存储器区域的数据可被多个总线主设备共用。存储器系统需要在不同总线主设备之间确保可共享存储器区域数据的一致性

5.36 处理器进入异常处理子程序之前保护现场需要把哪些寄存器的值保护起来？

[解析]

PSR，PC，LR，R0~R3，R12。

5.38 解释 Cortex-M 处理器的中断优先级分组机制。

[解析]

- 8 位的优先级配置寄存器分为两部分：分组优先级和（组内）子优先级。由于分组优先级和子优先级可以配置不同宽度的组合，因此可有效提高中断优先级配置的灵活度。
- 分组优先级即抢占优先级，高分组优先级的中断可打断低分组优先级的中断。
- 在相同的分组优先级下，具有更高子优先级的异常会被优先处理。

5.39 解释向量表重定位机制。

[解析]

向量表重定位使用 VTOR 指示向量表的位置，保存向量表相对于存储器的偏移量，处理器通过修改 VTOR 值修改向量表的起始位置，从而实现向量表重定位。

[补充 5.53] 以下关于 Cortex-M3/M4 堆栈说法正确的是（ ）

- I. 只支持满递减类型的栈
- II. 往栈内放置一个新的数据后，栈指针数值会增加
- III. 具有双栈的结构，有 MSP 和 PSP 两个堆栈指针
- IV. 非特权线程模式只能使用 PSP 而不能使用 MSP
- V. MSP 的值在上电前就需要存储在片内的 Flash 中
- VI. PSP 的值需要在初始化程序中进行设置
- VII. 栈空间只能放置在 4GB 空间的 SRAM 区
- VIII. 按照 AAPCS 要求，栈需要双字对齐

- A) I、II、III、IV
- B) II、III、IV、V
- C) III、IV、V、VI
- D) IV、V、VI、VII
- E) V、VI、VII、VIII
- F) 以上选项均不正确

[解析] C

- I. 只支持满递减类型的栈
- II. 往栈内放置一个新的数据后，栈指针数值会递减
- III. 具有双栈的结构，有 MSP 和 PSP 两个堆栈指针
- IV. 非特权线程模式只能使用 PSP 而不能使用 MSP
- V. MSP 的值在上电前就需要存储在片内的 Flash 中

- VI. PSP 的值需要在初始化程序中进行设置
- VII. 栈空间可以放置在 4GB 空间的 SRAM 区的 SRAM 或者 CODE 区的 SRAM 中
- VIII. 按照 AAPCS 要求，栈需要双字对齐

[补充 5.54] 以下关于 Cortex-M3/M4 存储器子系统说法正确的是（ ）

- I. 4GB 的存储器映射关系是固定不变的
 - II. 既可以配置为大端模式，也可以配置为小端模式
 - III. 所有 Cortex-M3/M4 的存储器访问指令都支持非对齐的数据传输
 - IV. 4GB 空间所有区域均可以采用位段（Bit-Band）操作方式来访问
 - V. 位段操作能保障“读→修改→写”的流程不被其他操作打断
 - VI. 处理器读存储器时，存储器系统会检查所访问区域的存储器访问属性
 - VII. 非特权线程访问内核私有区域可能会触发 MemManage Fault 异常
 - VIII. 处理器不会改变代码的执行顺序，因而不需要存储器屏障指令
- A) I、II、III、IV
B) II、III、IV、V
C) III、IV、V、VI
D) IV、V、VI、VII
E) V、VI、VII、VIII
F) A~E 均不满足题意

[解析] F

- I. 4GB 的存储器映射关系是固定不变的
- II. 既可以配置为大端模式，也可以配置为小端模式
- III. 并非所有 Cortex-M3/M4 的存储器访问指令都支持非对齐的数据传输
- IV. 4GB 空间中特定的区域可以采用位段（Bit-Band）操作方式来访问
- V. 位段操作能保障“读→修改→写”的流程不被其他操作打断
- VI. 处理器读存储器时，存储器系统会检查所访问区域的存储器访问属性
- VII. 非特权线程访问内核私有区域可能会触发 MemManage Fault 异常
- VIII. 处理器不会改变代码的执行顺序，但指令中的存储器访问操作完成的时间不一定和指令的顺序一致，仍可能用到存储器屏障指令

[补充 5.55] 以下关于 Cortex-M3/M4 中断/异常机制说法正确的是（ ）

- I. 特殊寄存器 PRIMASK 和 FAULTMASK 均用于中断屏蔽
 - II. 与 PRIMASK 不同的是，FAULTMASK 无需清理，当负责错误处理的异常处理程序返回时，会自动复位 FAULTMASK
 - III. 特殊寄存器 BASEPRI 采用了可伸缩的设计
 - IV. 每个异常都会处于激活、非激活、挂起、激活并挂起之一的某一个状态
 - V. 每个异常都有一个独立的优先级寄存器
 - VI. 异常的优先级寄存器只能由特权访问等级代码访问
 - VII. 中断优先级寄存器中数值约小，表示对应中断源的优先级约高
 - VIII. 多个相同优先级的异常同时发生时，处理器会先处理异常类型号低的异常
 - IX. 中断使能、中断禁止采用了 2 个独立的寄存器
 - X. 特定中断源的中断使能或禁止用寄存器中的 1 位来实现
- A) I、II、III、IV、V
B) II、III、IV、V、VI

- C) III、IV、V、VI、VII
- D) IV、V、VI、VII、VIII
- E) V、VI、VII、VIII、IX
- F) VI、VII、VIII、IX、X
- G) A~F 均不满足题意

[解析] G
只有 x 选项不正确

放到第 6 章？

5.5 ARM 指令集、Thumb 指令集和 Thumb-2 指令集之间的主要区别是什么？

第 8 章作业

PART1

-----PART1-----

第 8 章 part1 作业说明

- ☐ 以下以灰色双删除线标识的不做。
- ☐ *标识的为选作题（非教材上作业题，部分题涉及查阅芯片手册），选做题要求是 6 题中至少选 3 题做。

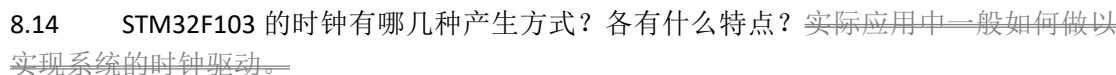
8.7 何为引脚功能复用？有何意义？~~如何实现，请以 STM32F103 为例，举例具体说明。~~

8.11 请设计一个 STM32 最小系统。

[解析] 包含模块：电源、时钟、复位、调试。

8.12 STM32F103 的复位电路有何功能？常见的复位方式有哪些？

[解析] 复位电路使芯片进入预定义的初始化状态（即主要寄存器被设置为缺省值）。
常见复位方式如下图所示，



STM32 的时钟来源：内部（HSI、LSI），外部（HSE、LSE）

低速时钟源有利于降低功耗;

使用内部时钟源无需外接谐振单元电路，但时钟稳定度差。

8.16 参照 STM32 的时钟树，请做以下思考，并解释 1) 设计成这种形式的主要目的和理由为何？2) 从左至右，相关时钟依次可分为大致哪几种？3) 时钟输出的使能有何意义，并以一个实例（如 TIM）说明一下大体流程。

【解析】基于寄存器、基于库、基于 OS、基于代码生成工具

优缺点可以从可移植性、可优化性、难易程度、硬件覆盖度等方面比较。

[解析] 安装 IDE、安装 DFP、安装标准外设库、代码编写、编译链接、利用 Debugger 下载至目标机、Run/Debug。

[补充 8A-1] 以下关于嵌入式系统开发说法正确的是 ()。

- | | |
|-------|--|
| IX. | 嵌入式生态系统一般指核心芯片、评估板、第三方硬件等资源 |
| X. | 嵌入式生态系统一般指硬件、软件、资料、服务等全部和产品相关的资源 |
| XI. | 嵌入式系统开发一般需要交叉编译 |
| XII. | 嵌入式系统开发必须要使用交叉编译 |
| XIII. | 嵌入式系统开发往往需要宿主机、目标机、Debugger |
| XIV. | OCD (On-Chip Debugging) 调试方式需要目标机硬件内置特定的支持电路 |
| XV. | OCD 调试方式下宿主机连接目标机需要特定的支持电路 |
- | | | | |
|----|-----------|----|----------|
| G) | I、II、III | I) | III、IV、V |
| H) | II、III、IV | J) | IV、V、VI |

K) V、VI、VII

L) A~E 均不符合题意

[解析] E
错误的描述包括，

- I. 嵌入式生态系统一般指核心芯片、评估板、第三方硬件等资源
- IV. 嵌入式系统开发必须要使用交叉编译

[补充 8A-2] * 对比意法半导体公司所有提供的四种库：STM32Snippets、Standard Peripheral Library、STM32Cube HAL、STM32Cube LL。

[解析] 抽象程度：HAL > SPL > LL > Snippets；
可移植性、可优化性、难易程度、产品就绪状态、硬件覆盖度如下

Offer		Portability	Optimization (Memory & Mips)	Easy	Readiness	Hardware coverage
	STM32Snippets		+++			+
	Standard Peripheral Library	++	++	+	++	+++
	HAL APIs	+++	+	++	+++	+++
	LL APIs	+	+++	+	++	++

[补充 8A-3] * 假设找出 class_show_8_2_stm32_clock.mp4 中的错误。录屏文件 <https://rec.ustc.edu.cn/share/e352c850-c582-11ec-93a4-b7910afe1e6b>。

[解析] HSE 还是 HSI，解说错误。

[补充 8A-4] *PC 寄存器和 SP 寄存器都保存的是“地址”，二者有何本质区别？

[解析] 一个存放代码地址，一个存放 Stack 指针（指向“数据”的地址）。

[补充 8A-5] *某 STM32F1 系列芯片启动后，地址“0x0000 0004”的存储单元内值为“0x0800 0800”，请问“0x0800 0800”代表了什么？

[解析] 复位异常处理程序的入口地址。

[补充 8A-6] *某 STM32F1 系列芯片启动后，地址“0x0000 003C”的存储单元内值为“0x0800 0800”，请问“0x0800 0800”代表了什么？

[解析] SysTick 异常处理程序的入口地址。

[补充 8A-7] *查阅 F103 系列芯片参考手册（BB 系统→课程资源→参考资料→STM32 资料→STM32F1 reference manual），说明如下 C 代码的功能。
(volatile unsigned long)(0x40021004) |= 0x28000000;

[解析] HSI 作为时钟输出。

Clock configuration register (RCC_CFGR)

0x40021004

Bits 26:24 MCO: Microcontroller clock output

Set and cleared by software.

0xx: No clock

100: System clock (SYSCLK) selected

101: HSI clock selected

110: HSE clock selected

111: PLL clock divided by 2 selected

[补充 8A-8] *简述 STM32F103 的启动过程。

[解析]

根据 BOOT0 和 BOOT1 引脚，选择启动存储器映射

从地址 0x00000000 处，取出栈顶指针值放入 MSP

从地址 0x00000004 处，取出复位异常服务程序入口地址放入 PC

执行复位异常服务程序 Reset_Handler()

[补充 8A-9] 已知 STM32F1 系列单片机有如下 GPIO 相关的寄存器，按照 p190 第 4 章 4.4.1 小节“4. I/O 接口结构”部分的分类方式，这些寄存器各自属于什么类型的端口寄存器？

寄存器名称	类型
Port configuration register low (GPIOx_CRL)	控制寄存器
Port configuration register high (GPIOx_CRH)	控制寄存器
Port input data register (GPIOx_IDR)	数据寄存器
Port output data register (GPIOx_ODR)	数据寄存器
Port bit set/reset register (GPIOx_BSRR)	数据寄存器
Port bit reset register (GPIOx_BRR)	数据寄存器
Port configuration lock register (GPIOx_LCKR)	控制寄存器
Alternate function I/O and debug configuration (AFIO)	控制寄存器

[补充 8A-10] 根据“stm32f10x.h”中的定义（如下），（1）说明代码行①操作了 4GB 存储空间的哪些存储单元（给出地址范围）的功能。（2）说明代码行①完成的功能。

//\stm32f10x.h

```
typedef struct{
    vu32 CRL;
    vu32 CRH;
```

```
vu32 IDR;

vu32 ODR;

vu32 BSRR;

vu32 BRR;

vu32 LCKR;

}GPIO_TypeDef;

#define GPIOA          ((GPIO_TypeDef *) GPIOA_BASE)
#define GPIOA_BASE      (APB2PERIPH_BASE + 0x0800)
#define APB2PERIPH_BASE (PERIPH_BASE + 0x10000)
#define PERIPH_BASE      ((uint32_t)0x40000000)
GPIOA->BSRR = 0x10000030;    //①
```

[解析]

(1) $0x4001\ 0810 \sim 0x4001\ 0813 = 0x40000000 + 0x10000 + 0x0800 + (4*4bytes)$

(2) BSRR 高 16bits 用于 Reset，低 16 位用于 Set，估①完成的功能是：复位（清零/Reset）PA12；同时置位（置 1/Set）PA4 和 PA5。

[补充 8A-11] 根据“stm32f10x.h”中的定义（如上题），(1)说明代码行①操作的 4GB 存储空间的哪些存储单元（给出地址范围）的功能。(2)说明代码行①和②完成的功能。

```
GPIOA->ODR |= 1<<7;          //①
GPIOA->ODR &= ~(1<<6));      //②
```

[解析]

(1) $0x4001\ 080C \sim 0x4001\ 080F = 0x40000000 + 0x10000 + 0x0800 + (3*4bytes)$

①和②PA7 置位（置 1/输出高电平）

(2) ①完成 PA7 置位（置 1/输出高电平）

②完成 PA6 清零（置 0/输出低平），等同于 $GPIOA_ODR \&= 0xBF(1011\ 1111B)$

[补充 8A-12] *根据“stm32f10x.h”中的定义（如上题），说明如下代码的功能。

```
GPIOA->ODR= *(volatile unsigned long*)(0x00000000);
```

[解析] 将主栈指针寄存器内的值写入 GPIOA 的输出数据寄存器，即根据 MSP 内的数据控制 PA0~PA15 的引脚输出电平。

-----PART1 END-----

PART2

-----PART2-----

第 8 章 part2 作业说明

- ❑ 以下以灰色双删除线标识的不做。
- ❑ *标识的为选作题（非教材上作业题，部分题涉及查阅芯片手册），选做题要求是 5 题中至少选 2 题做。

8.31 简述通用定时器的输入捕获过程。

[解析]

检测 TIMx_CHx 引脚信号，在信号发生跳变（比如上升沿/下降沿）时，将定时器计数值（TIMx_CNT）存放到对应的捕获/比较寄存器（TIMx_CCRx），完成一次捕获。
同时，还可以配置是否触发中断/DMA 等。

8.34 简述通用定时器的比较输出过程。

[解析]

当定时器计数值（TIMx_CNT）变动至与捕获/比较寄存器（TIMx_CCRx）值相等时，在 OCx 输出引脚输出特定信号，输出信号依据设置可能是：置位、复位、翻转或不变。
同时，还可以配置是否触发中断/DMA 等

8.36 利用 TIM2 中断，设计一个红绿灯系统，要求：红灯亮 5 秒钟，熄灭，切换绿灯亮 5 秒钟，熄灭，循环往复。给出硬件原理图，并编写代码实现。

[解析]

答案不唯一，合理即可。

void TIM2_IRQHandler() //中断服务子程序

```
{
    if(TIM_GetITStatus(TIM2,TIM_IT_Update!=RESET)
    {
        if(flag)
        {
            LED_GREEN_OFF(); //对 PB8 写 0
            LED_RED_ON(); //对 PA18 写 1
        }
        else
        {
            LED_RED_OFF(); //对 PA18 写 0
            LED_GREEN_ON(); //对 PB8 写 1
        }
    }
}
```

```
        flag = !flag; //反转 flag，这样下一次触发中断时亮不同的灯
    }
}
int main()
{
    int flag =0; //这是中断服务程序中指示当前亮的灯种类的变量
    LED_RED_Config(); //对 PA18 进行配置
    LED_GREEN_Config(); //对 PB8 进行配置
    NVIC_PriorityGroupConfig(NVIC_PriorityGroup_2); //设置中断优先级
    TIM2_Init(83999,4999); //TIM2 初始化，配置 arr 和 psc 使计数周期为 5s 并使能中断
    while(1)    {}
}
```

8.41 STM32F103 的中断系统共支持多少个异常？其中包括多少个内部异常和多少个可屏蔽的非内核异常中断？

[解析]

支持 84 个异常，包括 16 个内部异常和 68 个可屏蔽的非内核异常中断。

8.43 STM32F103 的外部中断/事件线是如何与端口映射的？

[解析]

除 3 个特定中断外，EXTI0~15 连接/映射方式如下：

任一端口 0 号引脚(如 PA0、PB0、…、PG0)映射到 EXTI0 上，

任一端口 1 号引脚(如 PA1、…、PG1)映射到 EXTI1 上…，

任一端口 15 号引脚(如 PA15、…、PG15)映射到 EXTI15 上。

同一时刻，只能有一个端口(即 A~G 中仅选一个)的 n 号引脚映射到 EXTI_n 上(n 为 0~15)

[补充 8A-13] 假设 TIMxCLK 为 72MHz，有如下代码。请问：（1）启动 TIM2 多长时间后 TIM2_CNT.CNT 数值为 720？（2）启动 TIM2 多长时间后 TIM2_ARR.ARR 与 TIM2_CNT.CNT 数值相同？

```
#include stm32f10x_tim.h

void main()
{
    //.....

    TIM_TimeBaseInitTypeDef my_timer;

    my_timer.TIM_Prescaler= 71999 ;

    my_timer.TIM_Period= 999 ;

    my_timer.TIM_CounterMode= TIM_CounterMode_Up ;

    TIM_TimeBaseInit (TIM2, & my_timer) ;
```

```
//.....  
}
```

[解析]

- (1) 时钟周期*(时钟周期个数 + 1) = $720/(72*10^6 / 8*10^4) = 0.8\mu s$
- (2) 定时周期(秒) = 预置数 * 分频系数 / 时钟频率
- Tout(s) = ((ARR+1) * (PSC+1)) / TIMxCLK = 80/72 = 10/9 秒

[补充 8A-14] *阅读教材 P558（或 Cortex™-M3 Devices Generic User Guide）关于 SysTick 各寄存器位定义的描述，分析如下代码。假设 HCLK 为 24MHz，请问：（1）代码行①、②、③的作用。（2）代码行④中的 ui_ticks 取值应为多少？

```
#define SysTick_CSR*((volatile unsigned int*) 0xE000E010) //控制寄存器  
#define SysTick_LOAD*((volatile unsigned int*) 0xE000E014) //重载寄存器  
#define SysTick_VAL*((volatile unsigned int*) 0xE000E018) //当前值寄存器  
#define SysTick_CALRB*((volatile unsigned int*) 0xE000E01C) //校准值寄存器  
SysTick_Init(unsigned int ui_ticks)  
{  
    SysTick_VAL = 0; //当前值寄存器清零  
    SysTick_CSR |= 0x4; //①  
    SysTick_CSR |= 0x2; //②  
    SysTick_CSR |= 0x1; //③  
    SysTick_LOAD = ui_ticks; //④  
}
```

[解析]

- 代码行①，CLKSOURCE=1，选择时钟源为 HCLK/8=4MHz（HCLK 参见教材 P469 图 8.21）
- 代码行②，TICKINT=1，使能中断
- 代码行③，ENABLE=1，使能 SysTick
- 代码行④， $(ui_ticks+1) \times (1/(3 \times 10^6)) = 10 \times 10^{-3} \rightarrow ui_ticks = 29999$

[补充 8A-15] *已知 STM32F407 实验箱上 LED 灯 D0 连接 PG10，开关 DIP0 连接 PE4。设计程序，实现 DIP0 开关拨动的时候触发中断，在中断服务子函数中控制灯的亮灭。请给出 main()函数代码和中断服务子程序代码。

[解析]

main()函数应包括 PG10 输出的 GPIO 初始化、PE4 输入的 GPIO 初始化、EXTI 初始化和 NVIC 初始化。

中断服务子程序应为 EXTI4_IRQHandler()。

8.50 简述 STM32F103 的 USART 数据接收/发送基本过程。

[解析]

- 发送过程：CPU 写 TDR 寄存器，TDR 数据存入发送移位寄存器后 SR.TXE 置 1，但此时发送并未结束，发送完毕时，SR.TC 置 1。同时，可配置产生 TC、TXE 等事件触发 USART 中断。
- 接收过程：RXD 引脚接收到串行数据，一位一位的数据移位进入接收移位寄存器，SR.RXNE 置 1，CPU 读 RDR 寄存器。同时，可配置产生 RXNE、NE 等事件触发 USART 中断。

[补充 8A-16] 解释如下寄存器位的含义。

寄存器位	含义
USART_SR.TXE	TDR 数据已经存入发送移位寄存器
USART_SR.TC	发送移位寄存器中各个比特发送完毕
USART_SR.RXNE	接收到数据，缓冲区非空

[补充 8A-17] *在 USART 收发数据的过程中，会产生 TC、TxE、RxNE 等不同事件，然而每个 USART 模块只对应一个中断向量。在中断服务程序中需要做什么处理？

[解析]

需要在中断服务子程序中读取相关标志位以判断产生的什么事件。

USART_SR[7] TXE: Transmit data register empty

USART_SR[6] TC: Transmission complete

USART_SR[5] RXNE: Read data register not empty

可能的处理代码形式如下：

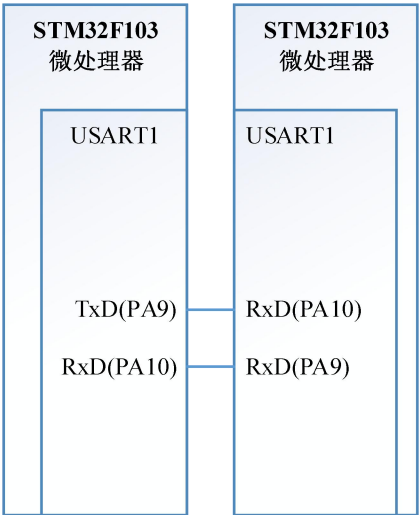
```
//例如，判断发送是否完成(TC)，返回值 SET 或 RESET，即 1/0
if (USART_GetFlagStatus(USART1, USART_FLAG_TC)==SET){//已发完，SR.TC==1};
//例如，判断是否接收到数据
if (USART_GetFlagStatus(USART1, USART_FLAG_RXNE) !=RESET)
{ //接收缓冲区非空，SR.RXNE==1 }
```

[补充 8A-18] *解释如下结构体各成员变量的含义。

```
typedef struct
{
    uint32_t USART_BaudRate;
    uint16_t USART_WordLength;
    uint16_t USART_StopBits;
    uint16_t USART_Parity;
    uint16_t USART_HardwareFlowControl;
    uint16_t USART_Mode;
} USART_InitTypeDef;
```

8.52 *设计一个简单的双机通信系统，要求：在两个 ARM 处理器之间，用 USART 实现，各自具有收/发缓冲内存区域，协议自定，结束条件自定。给出相关硬件连接图，并编写核心代码。

[解析]



答案不唯一，合理即可。以下给出的是单工方式，即一端只发，另一端只接收。

//发送端

```
int main (void) {
    unsigned char uc_send_ch = 0, uc_rcv_ch = 0;
    GPIOA_Config() ; //初始化 GPIOA
    USART1_Config (9600) ; //USART1 Init with 9600bps
    printf ("USART1 start...\n") ;
    while (1) {
        TIM2_Delay125MS();
        USART_SendData (USART1, uc_send_ch++) ; //发送字符
        printf("Send:0x%x\n",uc_send_ch);
    }
}
```

//接收端

```
int main (void) {
    unsigned char uc_send_ch = 0, uc_rcv_ch = 0;
    GPIOA_Config() ; //初始化 GPIOA
    USART1_Config (9600) ; //USART1 Init with 9600bps
    while (1) {
        TIM2_Delay125MS();
        if (USART_GetFlagStatus (USART1, USART_FLAG_RXNE) !=RESET){
            uc_rcv_ch = USART_ReceiveData(USART1); //接收字符
            printf("Receive: 0x%x\n",uc_rcv_ch);
        }
    }
}
```

-----PART2-----END

PART3

-----PART3-----

4.61 描述 I²C 总线协议中的状态，并画出状态转移图。

[解析]

总线空闲 I: SDA 与 SCL 均为高电平

启动数据传输 S: SCL 为高电平，SDA 负跳变

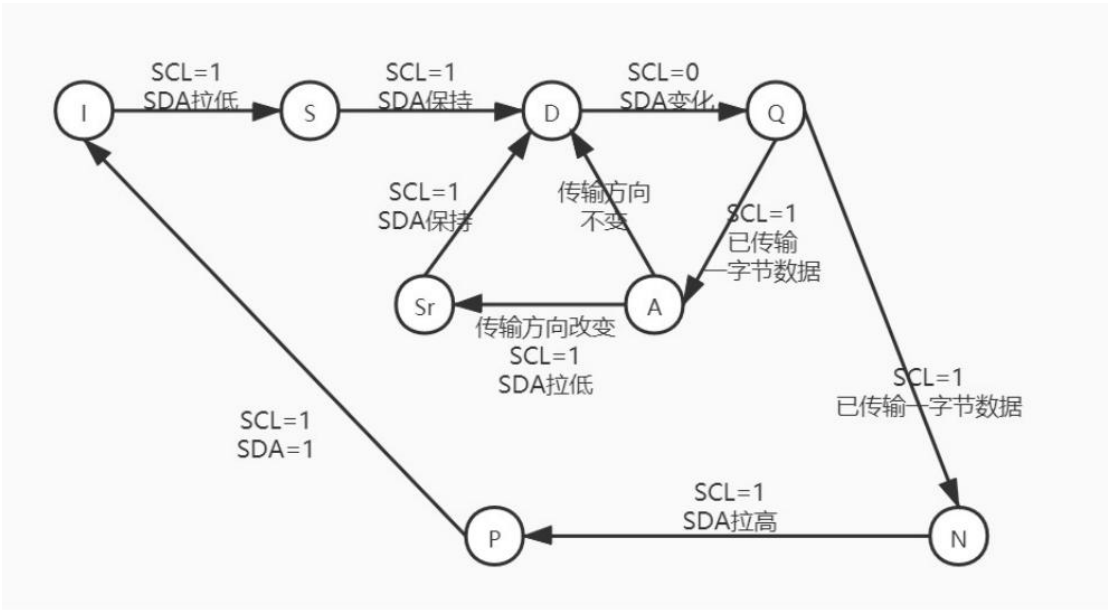
停止数据传输 P: SCL 为高电平，SDA 正跳变

重新启动 Sr: SCL 为高电平，SDA 负跳变

数据有效 D: SCL 为高电平期间，SDA 电平稳定

等待/ 数据无效 Q: SCL 保持低电平，此时可修改线上数据

应答 A /不应答 N: 接收方将 SDA 拉低发出 ACK 或释放 SDA 发出 NACK



[补充 4.64] 对比 “I²C 总线规范中定义的地址”、“访问存储器的地址”、“Cache 地址”、“I/O 端口地址” 有何不同？

[解析] 大致描述出以下信息即可，

- ❑ “I²C 总线规范中定义的地址” 是 I²C 器件地址，即器件的 ID，用于指示被访问的从器件。
- ❑ “访问存储器的地址” 是存储器内存储单元的 ID，用于指示拟访问的存储单元。
- ❑ “Cache 地址” 是 Cache 内存储单元的 ID。
- ❑ “I/O 端口地址” 是 I/O 接口中（控制、数据、状态）寄存器的 ID，用于指示被访问的寄存器。

[补充 4.66] 以下关于 SPI 总线说法正确的是（ ）

- VIII.

SPI 是一种全双工通信总线，即主设备读从设备、主设备写从设备可同时进行
- IX.

采用 SPI 通信时，主设备和从设备是共用时钟信号的
- X.

SPI 主设备通过 CS# 信号指示从设备数据传输的开始与结束
- XI.

SPI 是同步通信总线
- XII.

SPI 总线是串行通信总线
- XIII.

从设备向主设备发送数据既可以在时钟上升沿，也可以在时钟的下降沿
- XIV.

接收设备采样数据线的的时间总是滞后发送设备推送数据半个周期
- F)

仅 I、II、III、IV
- G)

仅 II、III、IV、V
- H)

仅 III、IV、V、VI
- I)

仅 IV、V、VI、VII
- J)

以上答案均不正确

[解析] E。
所有说法都是正确的

[补充 8A-19] *解释如下寄存器位的含义。

寄存器位	含义
SPI_CR2.TXEIE	
SPI_CR2.RXNEIE	
SPI_SR.TXE	
SPI_SR.RXNE	

[补充 8A-20] *在图 4.84 的不同时刻（如下图所示 T0 时刻、T1 时刻、T2 时刻），I²C 主设备和 I²C 从设备各产生哪些事件？从 I2C_SR1.SB、I2C_SR1.ADDR、I2C_SR1.STOPF、I2C_SR1.BTF、I2C_SR1.RXNE、I2C_SR1.TXE 中选择。

